

Documentação Inicial

Projeto 02

Engenharia de Sistemas Distribuídos

Prof. Raoni Kulesza

Bárbara Geovanna Alves Cavalcante

Fellype Dias Fontes

Thaís Melquíades Macêdo

Tobias Freire Numeriano

10 de julho de 2026

Sumário

1. Introdução.....	2
2. Visão Geral do Sistema.....	2
3. Diagramas C4.....	2
4. Decisões Arquiteturais (ADRs).....	3
ADR-001 – Armazenamento do histórico de eventos de suspeita.....	3
ADR-002 – Mecanismo de acionamento da quarentena automática.....	4
ADR-003 – Separação de leitura e escrita no painel administrativo.....	4
ADR-004 – Stack tecnológica do projeto.....	5
5. Stack Tecnológica.....	7
6. Padrões Arquiteturais.....	7
Padrão 1: Event Sourcing.....	7
Padrão 2: Feature Flag.....	8
Padrão 3: CQRS.....	9
Padrão 4: SAGA (Orchestration Pattern).....	9
7. Setup Inicial do Ambiente.....	10
7.1 Pré-requisitos.....	10
7.2 Estrutura dos microsserviços.....	10
7.3 CI.....	10
7.3.1 Fluxo de desenvolvimento.....	10
7.3.2 Pipeline de checagem.....	11
8. Divisão de Responsabilidades.....	11
8.1 Projeto 02: Documentação Inicial.....	11
8.2 Projeto 03: Implementação e Documentação Final.....	11
9. Próximos Passos.....	12

1. Introdução

Este documento apresenta a documentação inicial do projeto Risk Scoring Engine, desenvolvido como Projeto Final da disciplina de Engenharia de Sistemas Distribuídos (2026.1). O projeto implementa a POC 2 – Antifraude Mínimo Viável, cujo objetivo é construir um motor de risk scoring multifatorial capaz de detectar e colocar em quarentena jogadores suspeitos em tempo real.

O documento cobre as decisões arquiteturais tomadas até o momento, a stack tecnológica adotada, os padrões arquiteturais que serão aplicados, a estrutura do ambiente de desenvolvimento e a divisão de responsabilidades entre os integrantes da equipe.

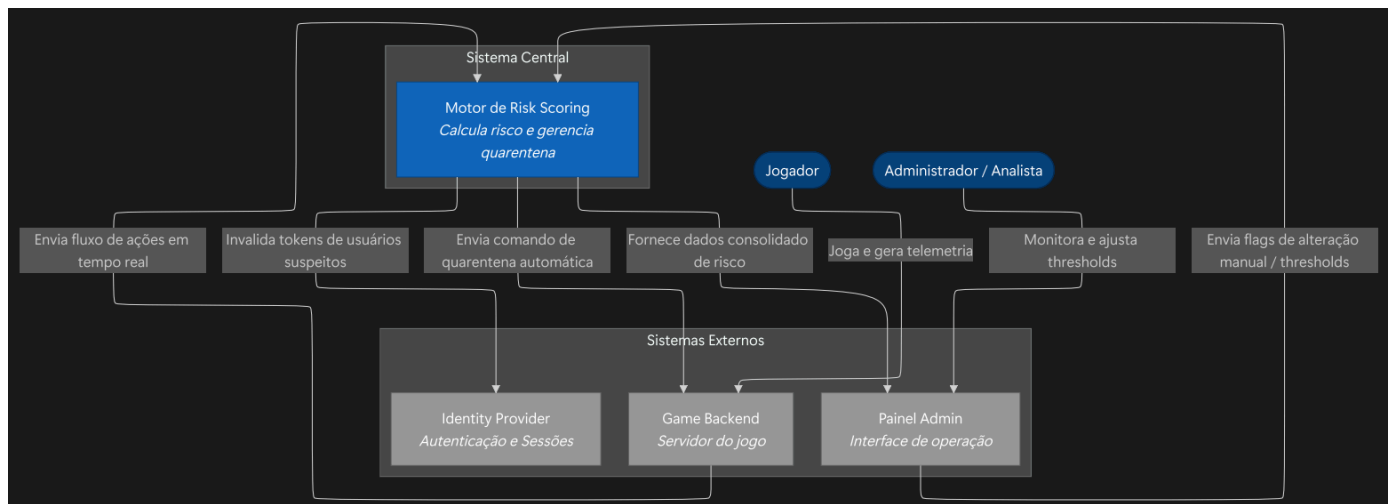
2. Visão Geral do Sistema

O Risk Scoring Engine é um sistema distribuído orientado a eventos que monitora o comportamento de jogadores em tempo real. A cada ação registrada, o sistema calcula uma pontuação de risco multifatorial considerando velocidade de resposta, padrão de escolhas, correlação entre contas e fingerprint de dispositivo. Quando essa pontuação ultrapassa um threshold configurável, o jogador é automaticamente colocado em quarentena e o painel administrativo é notificado.

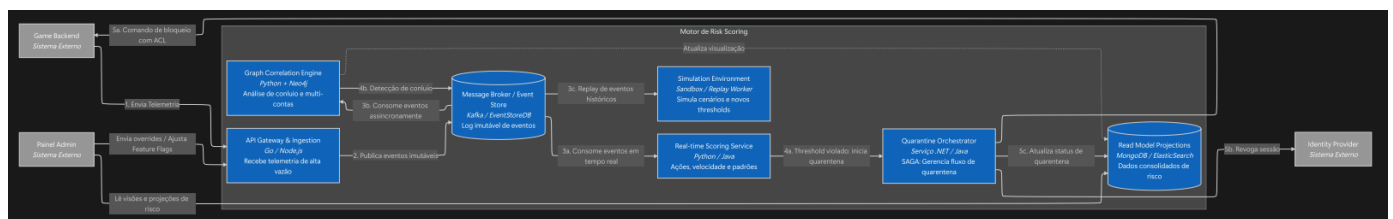
O sistema é composto por microsserviços independentes que se comunicam via eventos, garantindo desacoplamento entre os componentes e permitindo que cada parte evolua de forma autônoma.

3. Diagramas C4

Abaixo são exibidos os Diagramas C4 Model de níveis 1 e 2, de contexto e de containers, respectivamente. O de nível 1 tem como objetivo possibilitar uma visão geral inicial do sistema, considerando usuários e sistemas externos com os quais interage. O de nível 2 busca ampliar a análise dos limites do sistema, mostrando a estrutura geral da arquitetura de software e como as responsabilidades são distribuídas entre os containers.



C4 Model Nível 1: Contexto. Pode ser acessado no [link](#).



C4 Model Nível 2: Containers. Pode ser acessado no [link](#).

4. Decisões Arquiteturais (ADRs)

ADR-001 – Armazenamento do histórico de eventos de suspeita

Status: Proposto

Contexto:

O sistema de antifraude precisa registrar cada ação suspeita de um jogador (velocidade de resposta, múltiplos acessos, padrão de escolhas) para calcular a pontuação de risco e permitir auditoria posterior. Existem duas abordagens principais: armazenar apenas o estado atual de cada jogador (pontuação de risco + flags ativas), ou armazenar cada evento individualmente em ordem cronológica e derivar o estado a partir deles.

Decisão:

Adotaremos Event Sourcing – cada ação suspeita será registrada como um evento imutável com



timestamp, tipo e dados contextuais. O estado atual de um jogador será derivado pela leitura sequencial dos seus eventos.

Consequências:

- **Positivas:** histórico completo e auditável de por que um jogador foi marcado como suspeito; possibilidade de "rever" o passado com regras diferentes; alinhamento com os padrões recomendados pela POC 2.
- **Negativas:** maior volume de dados armazenados; reconstruir o estado atual exige processar todos os eventos do jogador, o que pode ser lento se o histórico for muito longo (mitigável com snapshots periódicos).

ADR-002 – Mecanismo de acionamento da quarentena automática

Status: Proposto

Contexto:

Quando a pontuação de risco de um jogador ultrapassa o threshold configurado, o sistema precisa acionar a quarentena automaticamente. Há duas formas de organizar isso: Orchestration (um serviço central coordena todos os passos – calcula a pontuação, decide a quarentena e notifica o painel admin) ou Choreography (cada serviço reage de forma independente a eventos publicados, sem um coordenador central).

Decisão:

Adotaremos o padrão de Orchestration via SAGA. Um serviço orquestrador receberá o evento de pontuação atualizada, decidirá se o threshold foi atingido e coordenará os passos seguintes: registrar a quarentena, bloquear o jogador e notificar o painel admin. Se qualquer etapa falhar, o orquestrador é responsável por desfazer as ações anteriores.

Consequências:

- **Positivas:** fluxo centralizado e mais fácil de debugar e auditar; comportamento de compensação (rollback) explícito e controlado; mais simples de entender e documentar para uma equipe pequena.
- **Negativas:** o orquestrador se torna um ponto central – se ele falhar, o fluxo de quarentena para; acoplamento maior entre o orquestrador e os demais serviços.

ADR-003 – Separação de leitura e escrita no painel administrativo

Status: Proposto

Contexto:

O painel administrativo precisa exibir em tempo real a lista de jogadores suspeitos, seus históricos e status de quarentena, ao mesmo tempo em que o sistema continua processando novos eventos e atualizando pontuações. Se leitura e escrita usarem o mesmo caminho e o mesmo banco de dados, consultas pesadas do painel podem competir com o processamento de eventos e degradar o desempenho geral.

Decisão:

Adotaremos CQRS (Command Query Responsibility Segregation), ou seja, leitura e escrita terão caminhos separados. Os comandos (registrar evento, atualizar pontuação, acionar quarentena) seguirão um fluxo de escrita dedicado. As consultas do painel admin lerão de uma projeção separada, atualizada de forma assíncrona a partir dos eventos armazenados.

Consequências:

- **Positivas:** o painel admin não impacta o desempenho do processamento de eventos; a projeção de leitura pode ser otimizada especificamente para as consultas do painel; alinhamento natural com Event Sourcing (ADR-001).
- **Negativas:** a projeção de leitura pode estar ligeiramente desatualizada em relação aos eventos mais recentes (consistência eventual); maior complexidade de implementação, uma vez que são dois modelos de dados para manter.

ADR-004 – Stack tecnológica do projeto

Status: Proposto

Contexto:

A POC 2 exige suporte a Event Sourcing, CQRS, SAGA/Orchestration, Circuit Breaker, Feature Flag e Anti-corruption Layer. A equipe tem maior familiaridade com Python e Java, e precisa de uma stack que viabilize esses padrões sem infraestrutura excessivamente complexa dentro do prazo disponível.

Opções Consideradas

Opção A: Python (FastAPI + Celery + SQLAlchemy)

- Familiaridade da equipe com a linguagem.
- Ecossistema menos maduro para microsserviços e padrões de sistemas distribuídos em comparação ao Java.

- Suporte a Circuit Breaker, SAGA e Event Sourcing existe, mas é menos consolidado e exige mais configuração manual.

Opção B: Java 21 + Spring Boot 3.3 (escolhida)

- Ecossistema extremamente maduro para microsserviços e padrões distribuídos.
- Spring Kafka para mensageria com Event Sourcing; Resilience4j para Circuit Breaker; estrutura de módulos que facilita separação de comandos e consultas (CQRS).
- Orientação a Objetos como paradigma central facilita organização do código entre a equipe.

Decisão:

Adotaremos Java 21 + Spring Boot 3.3 como base, com os seguintes componentes:

- Kafka como broker de eventos: escolhido por oferecer log de eventos ordenado e particionado, requisito do Event Sourcing.
- PostgreSQL como banco de escrita (write model): persistência transacional e histórico de eventos.
- Redis como banco de leitura (read model CQRS): consultas rápidas de score e status de quarentena sem reprocessar o log a cada requisição.
- Docker + Docker Compose para containerização e execução portátil de toda a infraestrutura.
- Prometheus + Grafana como diferencial de observabilidade, implementado se houver tempo disponível.

Consequências

Positivas:

- Todos os padrões exigidos pela POC 2 têm suporte nativo ou amplamente documentado no ecossistema Spring.
- Docker Compose simplifica o onboarding: qualquer membro roda toda a infraestrutura com um único comando.
- PostgreSQL + Redis como modelos separados de escrita/leitura implementa CQRS de forma natural e sem acoplamento.

Negativas / Riscos:

- Java tem mais boilerplate do que Python: o tempo de setup inicial de cada microserviço é maior.
- Kafka adiciona complexidade operacional; falhas na configuração do broker podem bloquear o desenvolvimento de outros serviços que dependem de eventos.
- Prometheus + Grafana ficam como item condicional, o que significa que podem não entrar na entrega final.

5. Stack Tecnológica

Acabamos optando por uma stack voltada ao Java, por ser uma linguagem que possui um ecossistema extremamente maduro para microserviços e sistemas distribuídos como um todo. Além de permitir uma melhor organização de código, por ter como propósito a Orientação a Objeto.

- **Linguagem/Framework: Java 21 + Spring Boot 3.3:** Ecossistema maduro e altamente suportado pela comunidade para microserviços e os padrões arquiteturais exigidos. Spring Kafka para mensageria, Resilience4j (via 'spring-boot-starter' oficial da comunidade) para Circuit Breaker, e estrutura de módulos que facilita separar comandos de consultas (CQRS).
- **Kafka:** Broker de eventos. Optamos pelo Kafka tradicional porque o Event Sourcing depende de um log de eventos ordenado e particionado.
- **PostgreSQL + Redis:** O Postgres será usado como banco de persistência para dados transacionais e histórico (write model). O Redis atuará como read model (CQRS) para consultas rápidas de score e status de quarentena sem reprocessar o log de eventos a cada requisição.
- **Prometheus + Graphana:** Usados para monitorar métricas e saúde dos microserviços. É algo que implementaremos como um diferencial caso sobre tempo.
- **Docker:** Ajudará a portabilizar e rodar os microserviços em diferentes ambientes, inclusive contribuindo para um CI/CD mais simples.

6. Padrões Arquiteturais

Padrão 1: Event Sourcing

Apesar de complexo, vale muito no contexto do projeto para aumentar o máximo possível a segurança e a confiabilidade da ferramenta como um todo, mantendo um nível considerável de



auditoria. Outro fator importante, essa rastreabilidade oferecida pelo Event Sourcing é extremamente importante em sistemas antifraude. Pois ele não serve apenas para auditoria posterior, mas permite que o motor de risco reconstrua o histórico exato de ações do jogador para identificar anomalias.

Prós:

- Rastreabilidade Absoluta: Fornece um histórico imutável completo de todas as ações dos usuários, facilitando investigações complexas de fraude.
- Simulação de Cenários: Permite fazer o replay (reprodução) de eventos passados para testar e calibrar novas regras de detecção de bots ou conluíus coordenados sem afetar a produção.
- Performance de Escrita: Como os eventos são apenas inseridos (append-only), a gravação das ações do jogo no banco é extremamente rápida.

Contras:

- Curva de Aprendizado Elevada: Requer uma mudança de mentalidade da equipe, já que o estado atual do sistema não fica exposto de forma convencional em uma tabela.
- Evolução do Schema: Alterar a estrutura de eventos antigos que já foram gravados (ex: adicionar campos a um evento de 6 meses atrás) pode se tornar complexo.

Padrão 2: Feature Flag

Resolve a necessidade de ativar ou desativar regras antifraude, limiares (thresholds) e algoritmos de detecção sem necessidade de realizar um novo deploy, facilitando testes, ajustes operacionais e respostas rápidas a mudanças no comportamento das fraudes. Ou seja, o Admin não precisa apenas ligar/desligar o sistema; ele pode alterar o peso do device fingerprint ou desativar temporariamente a validação de mult celular se houver um falso positivo em massa, tudo em tempo real e sem downtime.

Prós:

- Agilidade Operacional: Permite mitigar ataques de fraude emergentes alterando pesos e regras instantaneamente no Painel Admin.
- Segurança em Atualizações: Caso uma nova regra de detecção comece a gerar falsos positivos bloqueando jogadores legítimos, ela pode ser desativada em milissegundos.
- Testes em Produção: Facilita a validação de novas regras com grupos específicos de jogadores antes de liberar para toda a base.

Contras:

- Débito Técnico: Se as flags antigas não forem removidas do código após os testes, o sistema acumula ramificações condicionais (if/else) difíceis de manter.
- Gerenciamento de Estado: Exige uma estratégia de cache eficiente (como Redis) para que a verificação constante das flags não introduza latência no motor de tempo real.

Padrão 3: CQRS

Trabalha com a parte de desempenho. Considerando que o sistema será em tempo real, a velocidade de processamento também será um fator decisivo para o sucesso do motor. E como haverá o sistema do admin e do usuário usando tipos de consultas diferentes, o CQRS é uma ótima alternativa para dar mais eficiência ao uso dos recursos. Por exemplo, o lado de Escrita (Command) recebe os eventos brutos do jogo e roda as regras rápidas de score. O lado de Leitura (Query) consome esses mesmos eventos de forma assíncrona para montar as visões ricas que o Admin precisa (ex: gráficos de correlação entre contas suspeitas). Isso garante que uma busca pesada do Admin não cause lentidão no fluxo do jogo.

Prós:

- Escalabilidade Independente: A infraestrutura de ingestão de eventos do jogo pode crescer separadamente da infraestrutura de relatórios do Painel Admin.
- Otimização de Banco de Dados: Permite usar modelos relacionais ou NoSQL otimizados especificamente para buscas complexas do Admin, sem travar as escritas em tempo real.

Contras:

- Consistência Eventual: Como o lado da leitura é atualizado de forma assíncrona, pode haver um pequeno atraso de segundos até que um evento ocorrido no jogo apareça renderizado nos gráficos do Painel Admin.
- Duplicação de Modelos: Exige a manutenção de códigos separados para comandos (escritas) e consultas (leitura).

Padrão 4: SAGA (Orchestration Pattern)

Adotado para gerenciar o mecanismo de acionamento da quarentena automática sempre que a pontuação de risco do jogador ultrapassar o threshold. Um serviço orquestrador centralizado receberá o evento de pontuação atualizada, decidirá se o limite foi atingido e coordena de forma sequencial e controlada os passos seguintes: registrar a quarentena, bloquear o jogador no jogo e notificar o painel admin. Caso ocorra falha em qualquer uma dessas etapas, o orquestrador acionará ações de compensação para desfazer os passos anteriores, garantindo a integridade dos dados.

Prós:

- Fluxo Centralizado e Controlado: Torna o fluxo de quarentena mais simples de entender, documentar, auditar e debugar, o que é ideal para equipes pequenas.
- Resiliência com Rollback Explícito: Se o bloqueio do jogador falhar no meio do caminho, o sistema sabe exatamente como reverter os passos anteriores automaticamente para evitar estados inconsistentes.

Contras:

- Ponto Central de Falha (SPOF): Se o serviço orquestrador ficar fora do ar, todo o fluxo de quarentena e punição automática do sistema é interrompido.
- Acoplamento: Cria uma dependência direta (acoplamento maior) entre o serviço orquestrador e os subsistemas de bloqueio, quarentena e painel.

7. Setup Inicial do Ambiente

7.1 Pré-requisitos

O setup inicial consiste primeiramente em:

1. Ter um repositório no GitHub
2. Ter Docker instalado

Com isso, a infraestrutura e microsserviços do projeto poderão ser buildados e rodados em containers docker a partir da configuração de um arquivo `docker-compose.yml`, com o comando `docker compose up`.

7.2 Estrutura dos microsserviços

Em cada pasta de microsserviço, que serão feitos em Java com Spring Boot, haverá:

1. Arquivo Dockerfile com instruções para build da imagem e construção do container docker que o `docker-compose.yml` irá usar.
2. Arquivo pom.xml com metadados e dependências do microsserviço (arquivo relacionado a projetos Spring Boot)
3. Código fonte do serviço

7.3 CI

7.3.1 Fluxo de desenvolvimento

Também é necessário implementar um fluxo de CI, que será feito através do GitHub. Para isso, primeiro definimos um fluxo de desenvolvimento:

1. 'Dev x' clona o repositório do projeto (`git clone`)
2. Muda para branch de trabalho (`git checkout -b <nome_branch>`)
3. Faz alterações nessa branch (`git add && git commit`)
4. Envia as alterações da branch de trabalho para o repositório remoto (`git push`)
5. Abre Pull Request para a branch 'dev'
6. Resolve conflitos se houver
7. 'Dev y' revisa Pull Request, se ok, realiza o merge na branch 'dev'

A branch 'main' (principal) fica reservada para ser utilizada no deploy. Além disso, é importante que os devs sempre mantenham suas branches locais atualizadas para evitar conflitos (`git pull`).

7.3.2 Pipeline de checagem

No projeto, é definido um workflow que usa o GitHub Actions para automatizar validações sempre que um PR é aberto, garantindo que qualquer código que deseje ser mergeado siga os padrões de qualidade estabelecidos. O(s) arquivo(s) que definem esse workflow podem ser encontrados no repositório em [.github/workflows/*](#), e devem fazer o seguinte quando alguém abre um Pull Request:

1. Roda linter nos microsserviços para garantir padrão de código
2. Faz o build de cada microsserviço e roda os testes presentes em cada um (se houver)

Quando alguma etapa desse workflow não finaliza com sucesso, quer dizer que algo no Pull Request precisa ser ajustado. Logo, o Pull Request fica bloqueado até que o Dev responsável corrija a origem do problema.

8. Divisão de Responsabilidades

8.1 Projeto 02: Documentação Inicial

Integrante	Responsabilidade
Bárbara	Architecture Decision Records (ADR-001 a ADR-004)
Fellype	Identificação e descrição dos padrões arquiteturais
Thaís	Diagramas C4 (níveis 1 e 2)
Tobias	Stack tecnológica, justificativas e setup inicial do ambiente
Todos	Introdução, visão geral, próximos passos e apresentação

8.2 Projeto 03: Implementação e Documentação Final

Integrante	Responsabilidade
Bárbara e Fellype	Serviço de ingestão de eventos
Bárbara	Serviço de scoring
Tobias	Serviço de quarentena (orquestrador SAGA)
Fellype	Serviço de leitura / painel admin



Thaís e Tobias	Infraestrutura e CI/CD
Thaís	Simulação de fraude e testes
Todos	Documentação final e vídeo

9. Próximos Passos

Com a documentação inicial estabelecida, as próximas etapas do projeto são:

- Implementação dos microsserviços conforme a arquitetura definida nos diagramas C4 e nas ADRs
- Desenvolvimento dos cenários de simulação de fraude (bot, multi-conta, conluio coordenado)
- Integração entre os serviços via Kafka e validação do fluxo de quarentena end-to-end
- Coleta de métricas e elaboração do plano de testes de carga e resiliência
- Preparação da documentação final e do vídeo para a entrega do Projeto 03